```
---
title: "SQL Report including load_data.sql and Querying_and_Manipulation.sql questions"
author: "Jonathan Dale"
date: "2026-03-17"
output: pdf
Title: CS 504 project stage 2 load_data.sql, Querying_and_Manipulation.sql
git: https://github.com/jonathan-dale/CS504
---

-- First load the data
-- To load data into local database, from the same directory, or use the full path.
-- psql -h 127.0.0.1 -p 5432 -U postgres -d cs-504 -f load_data.sql


-- ============================================
-- 1. Drop tables (optional, for a clean reload)
-- ============================================
DROP TABLE IF EXISTS authors        CASCADE;
DROP TABLE IF EXISTS catalog_tbl    CASCADE;
DROP TABLE IF EXISTS genre          CASCADE;
DROP TABLE IF EXISTS member_tbl     CASCADE;
DROP TABLE IF EXISTS staff          CASCADE;
DROP TABLE IF EXISTS material       CASCADE;
DROP TABLE IF EXISTS authorship     CASCADE;
DROP TABLE IF EXISTS borrow         CASCADE;
-- ============================================
-- 2. Create tables
--    table_name: attributes
--    authors:author_id,name,nationality
--    authorship:authorship_id,author_id,material_id
--    borrow:borrow_id,material_id,member_id,staff_id,borrow_date,due_date,return_date
--    catalog:catalog_id,name,location
--    genre:genre_id,name,description
--    material:material_id,title,publication_date,catalog_id,genre_id
--    member_tbl:member_id,name,contact_info,join_date
--    staff:staff_id,name,contact_info,job_title,hire_date
-- ============================================

-- Notice: the first 5 tables have no dependencies
CREATE TABLE authors (
    author_id    INTEGER PRIMARY KEY,
    name         TEXT NOT NULL,
    nationality  TEXT
);

CREATE TABLE catalog_tbl (
    catalog_id   INTEGER PRIMARY KEY,
    name         TEXT NOT NULL,
    location     TEXT
);

CREATE TABLE genre (
    genre_id     INTEGER PRIMARY KEY,
    name         TEXT NOT NULL,
    description  TEXT
);

CREATE TABLE member_tbl (
    member_id    INTEGER PRIMARY KEY,
    name         TEXT NOT NULL,
    contact_info TEXT,
    join_date    DATE
);
```

```
CREATE TABLE staff (
    staff_id     INTEGER PRIMARY KEY,
    name         TEXT NOT NULL,
    contact_info TEXT,
    job_title    TEXT,
    hire_date    DATE
);

-- Notice: Material table must be created before authorship and borrow

CREATE TABLE material (
    material_id      INTEGER PRIMARY KEY,
    title            TEXT NOT NULL,
    publication_date INTEGER,
    catalog_id       INTEGER REFERENCES catalog_tbl(catalog_id),
    genre_id         INTEGER REFERENCES genre(genre_id)
);

CREATE TABLE authorship (
    authorship_id INTEGER PRIMARY KEY,
    author_id     INTEGER REFERENCES authors(author_id),
    material_id   INTEGER REFERENCES material(material_id)
);

CREATE TABLE borrow (
    borrow_id    INTEGER PRIMARY KEY,
    material_id  INTEGER REFERENCES material(material_id),
    member_id    INTEGER REFERENCES member_tbl(member_id),
    staff_id     INTEGER REFERENCES staff(staff_id),
    borrow_date  DATE,
    due_date     DATE,
    return_date  DATE
);


-- ============================================
-- 3. Load data from CSV files
--    Paths are relative
--    Keep same order as creation
-- ============================================

\copy authors FROM './Data/Authors.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',', NULL
'', QUOTE '"');
\copy catalog_tbl FROM './Data/Catalog.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',',
NULL '', QUOTE '"');
\copy genre FROM './Data/Genre.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',', NULL '',
QUOTE '"');
\copy member_tbl FROM './Data/Member.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',', NULL
'', QUOTE '"');
\copy staff FROM './Data/Staff.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',', NULL '',
QUOTE '"');
\copy material FROM './Data/Material.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',', NULL
'', QUOTE '"');
\copy authorship FROM './Data/Authorship.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',',
NULL '', QUOTE '"');
\copy borrow FROM './Data/Borrow.csv' WITH (FORMAT csv, HEADER true, DELIMITER ',', NULL '',
QUOTE '"');




-- ==================================
-- Next, Querying_and_Manipulation.sql
-- ==================================
```

```
---
title: "Querying_and_Manipulation.sql"
author: "Jonathan Dale"
date: "2026-03-17"
output: pdf
Title: CS 504 project stage 2 Querying_and_Manipulation.sql
---


-- question 1
-- List the names and joined dates of all members who registered in the year 2018.
select mt.join_date, mt."name"
from
    member_tbl mt
where mt.join_date between date '2018-01-01' and date '2018-12-31';

-- question 2
-- Retrieve the names of members along with the total count of books they have borrowed.
select
    mt.name,
    count(b.material_id ) as total_books_borrowed
from
        member_tbl mt
left join
        borrow b ON mt.member_id  = b.member_id
group by mt.member_id, mt."name";

-- question 3
-- Calculate the average borrowing duration (in days) for all borrowed books.
-- this one is tricky.....
select
        (b.return_date - b.borrow_date )/count(b.borrow_date) as average_borrow_time
from
        borrow b
group by b.return_date, b.borrow_date

select
         b.material_id,
         count(b.material_id) is distinct
        (b.return_date - b.borrow_date ) as average_borrow_time,
from
        borrow b
group by b.return_date, b.borrow_date,  b.material_id


-- sum (return - borrow) / number of times borrored

select count(b.material_id) distinct
from borrow b;


select b.borrow_id
from borrow b
where b.material_id is

----------------


-- question 4
-- Identify the author who has the highest number of books in the library, including their
name
-- and the total book count.
select
    au.name,
```

```
    count(distinct a.material_id) as total_book_count
from authors au
join authorship a
  on au.author_id = a.author_id
group by au.author_id, au.name
order by total_book_count desc
limit 1;


-- question 5
-- List all authors along with the titles of the books they have written.
select
    a.name as author,
    m.title as book_title
from authors a
left join authorship au
  on au.author_id = a.author_id
left join material m
  on m.material_id = au.material_id
order by a.name, m.title;


-- question 6
-- Determine the book that has been borrowed the most times.
select
    m.title,
    count(*) as times_borrowed
from borrow b
join material m
    on m.material_id = b.material_id
group by m.material_id, m.title
order by times_borrowed desc
fetch first 1 row with ties;

-- question 7
-- Retrieve the list of members who have borrowed books but have not yet returned them
-- (return_date is NULL).
select
    mtb.name
from member_tbl mtb
join borrow b
  on b.member_id = mtb.member_id
where b.return_date is null;

-- The check below shows inconsistent behavior....
select
    count(mtb."name")
from member_tbl mtb
join borrow b
  on b.member_id = mtb.member_id
where b.return_date is null;
-- returns 28

select
    count(member_id)
from borrow
where return_date is null;
-- returns 22

select
    count(mtb."name")
from member_tbl mtb
join borrow b
  on b.member_id = mtb.member_id
where b.return_date is null;
```

```
-- returns 22, correct solution!


-- question 8
-- Identify the staff member who has processed the highest number of book borrow
transactions.
select
    s.name,
    count(b.borrow_id) as transactions_processed
from staff s
join borrow b
  on b.staff_id = s.staff_id
group by s.staff_id, s.name
order by transactions_processed desc
limit 1;


-- question 9
-- Find the most borrowed book genre based on borrowing records.
select
    g.name as genre,
    count(*) as times_borrowed
from borrow b
join material m
  on m.material_id = b.material_id
join genre g
  on g.genre_id = m.genre_id
group by g.genre_id, g.name
order by times_borrowed desc
limit 1;

-- question 10
-- List all books that are overdue (i.e., where the due date has passed but the return_date
is
-- still NULL).
select
        m.title
from material m
join borrow b
  on m.material_id = b.material_id
where b.return_date is null
  and b.due_date < current_date;



```